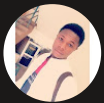


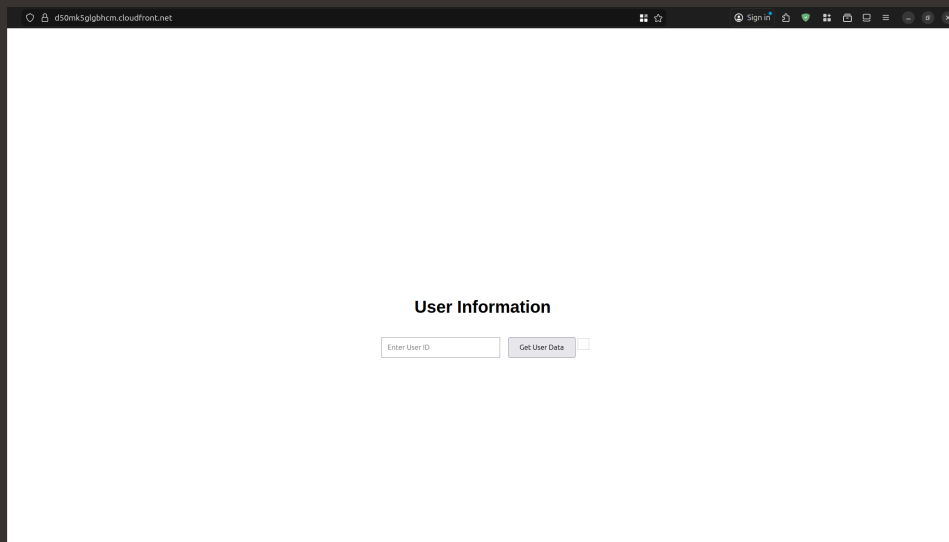


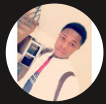
[nextwork.org](https://nextwork.org)

# Website Delivery with CloudFront



Ahmed Tetteh





# Introducing Today's Project!

In this project, I will demonstrate how to present the front facing part of a website on Amazon services using S3 and CloudFront. By doing this, we can deliver faster website access to our users, using CloudFront's edge locations.

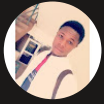
## Tools and concepts

Services I used were CloudFront and Amazon S3. Key concepts I learnt include content delivery network (CDN) with CloudFront, and how to host a static website on S3. Also, I learnt the importance of the role CloudFront performs for website hosting.

## Project reflection

This project took me approximately 2 hrs (including error troubleshooting on both the CloudFront and S3 bucket side). The most challenging part was creating the using the OAC apporriately to give CloudFront the right permissions it needs to access the bucket. It was most rewarding to my website hosted on both S3 and CloudFront.

I chose to do this project today because website hosting is one of the necessary tools to have in your arsenal as a Cloud developer or DevOps Engineer.

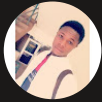


## Set Up S3 and Website Files

I started the project by creating an S3 bucket to store my website files. I can't use CloudFront for this task because it's a CDN service, not a storage solution. It simply caches and distributes your contents closer to end users through edge locations.

The three files that make up my website are `index.html`, which is the main file for the website that organizes the texts, pictures, and videos on the webpage, `style.css` organizes the visual appearance of the website's HTML elements (font style, size, color and more) and finally, the `script.js`, which is the javascript file that adds interaction to the webpage. It includes instructions on how things should move or change on the website. Without it, the website would just look pretty or beautiful, but with no interactive capabilities.

I validated that my website files work by opening the `index.html` file on my preferred browser of choice.



nextwork-three-tier-fap

Info

Objects

Metadata

Properties

Permissions

Metrics

Management

Access Points

Objects (3)

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Find objects by prefix

Name

Type

Last modified

Size

Storage class

index.html

html

March 5, 2026, 22:40:40 (UTC+09:00)

564.0 B

Standard

script.js

js

March 5, 2026, 22:40:41 (UTC+09:00)

680.0 B

Standard

style.css

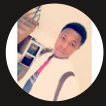
css

March 5, 2026, 22:40:41 (UTC+09:00)

447.0 B

Standard



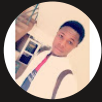


# Exploring Amazon CloudFront

Amazon CloudFront is a content delivery network, which means a service that caches content and speeds up the distribution of such contents faster to users. Businesses and developers use CloudFront because it enables them to cache their contents on multiple servers around the world, closer to their end-users, reducing latency and delivering faster access to them.

To use Amazon CloudFront, you set up distributions, which are simply settings to control how my contents or assets are delivered to end-users around the world. I set up a distribution for Amazon S3 bucket. The origin where my content lives, i.e, the storage location of my website files.

My CloudFront distribution's default root object is the index.html file. This means that when users visit my website, they will see the contents of my index.html file.



### Specify origin

#### Origin type

Your origin is where your content (such as a website or app) lives. CloudFront works with AWS-based origins and origins hosted on other cloud providers.

**Origin type**

☒ **Amazon S3**  
Deliver static assets like files and images, statically generated websites or single page applications (SPAs).

☐ **Elastic Load Balancer**  
Deliver applications hosted behind ELB such as dynamic websites, web services, and APIs.

☐ **API Gateway**  
Deliver API endpoints for REST APIs hosted on API Gateway.

☐ **Elemental MediaPackage**  
Deliver end-to-end live events or video on demand (VOD).

☐ **VPC origin**  
Deliver applications and content hosted within private VPCs, such as EC2 instances and Application Load Balancers.

☐ **Other**  
Refer to any AWS or non-AWS origin through its publicly resolvable URL.

#### Origin

**S3 origin**  
Choose an AWS origin, or enter your origin's domain name. [Learn more](#)

[Browse S3](#)

**Origin path - optional**  
The directory path within your origin where your content is stored. [Learn more](#)

#### Settings

CloudFront provides default origin and cache settings based on what origin you selected. [View default settings for S3](#)

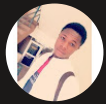
**Allow private S3 bucket access to CloudFront** [Info](#)  
CloudFront will update your S3 bucket policy to allow CloudFront to access your S3 bucket. The policy allows CloudFront to access the bucket only when the request is on behalf of the CloudFront distribution that contains the S3 origin.

☒ **Allow private S3 bucket access to CloudFront - Recommended**

**Origin settings**  
Origin settings control how CloudFront connects to the specified origin.

☒ **Use recommended origin settings**

☐ **Customize origin settings**

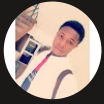


## Handling Access Issues

When I tried visiting my distributed website, I ran into an access denied error because CloudFront hasn't been given the permission to access the S3 bucket yet.

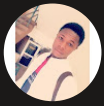
My distribution's origin access settings were set to OAC, i.e, access is restricted to only CloudFront, making CF access the contents of my bucket. An access error still occurred because this setting does not reflect in my bucket policies. So, though, in my CloudFront distribution, I have stated for only CloudFront to access my S3 bucket through the OAC, the same policy hasn't been created in my S3 bucket policies,

OAC is a special user or role that CloudFront uses to access the objects in my bucket. It helps keep my bucket private, but still accessible by CloudFront. It can also provide control over how CloudFront accesses contents.



A screenshot of a web browser window. The address bar shows the URL `d2mez740qxaqfk.cloudfront.net`. Below the address bar, a message states: "This XML file does not appear to have any style information associated with it. The document tree is shown below." Below this message, an XML document tree is displayed with the following content:

```
<?xml version="1.0" encoding="UTF-8"?>
<Error>
  <Code>AccessDenied</Code>
  <Message>Access Denied</Message>
</Error>
```



# Updating S3 Permissions

Once I set up my OAC, I still needed to update my bucket policy because my S3 bucket by default, is private. So, I need to create a policy that allows CloudFront to access my bucket.

Creating an OAC automatically gives me a policy I could copy, which grants CloudFront the right permissions it needs to access my S3 bucket.

**Edit bucket policy** info

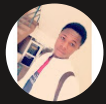
**Bucket policy**

The bucket policy, written in JSON, provides access to the objects stored in the bucket. Bucket policies don't apply to objects owned by other accounts. [Learn more](#)

**Bucket ARN**  
 arn:aws:s3:::nextwork-three-tier-fap-1

**Policy**

```
1  {
2    "Version": "2008-10-17",
3    "Id": "PolicyForCloudFrontPrivateContent",
4    "Statement": [
5      {
6        "Sid": "AllowCloudFrontServicePrincipal",
7        "Effect": "Allow",
8        "Principal": {
9          "Service": "cloudfront.amazonaws.com"
10       },
11       "Action": "s3:GetObject",
12       "Resource": "arn:aws:s3:::nextwork-three-tier-fap-1/*",
13       "Condition": {
14         "StringEquals": {
15           "AWS:SourceArn": "arn:aws:cloudfront::007399901516:distribution/E1MQQWBQ3V1RGZ"
16         }
17       }
18     ]
19   }
20 }
```



## S3 vs CloudFront for Hosting

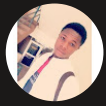
For my project extension, I'm comparing the performance and security associated with accessing my website directly from the S3 bucket, compared to serving from CloudFront. I initially had an error with static website hosting because public access wasn't enabled yet the bucket. This means any anyone should be able to access the website directly from the bucket.

I tried resolving this by disabling the "Block public access" button I still ran into an error because that simply stopped blocking all block access, but didn't enable public access for anyone. What will do that is the bucket policy!

I could finally see my S3 hosted website when I added a block within my bucket policy, enabling public access to anyone for any object within the bucket. This worked because the "Block public access" button is just the first layer of security on the bucket that either allows or blocks all public access to the bucket. The second and most vital layer of security that grants a more granular detail in control is the "Bucket policies". This allows you to state who can access the bucket and what they are allowed to perform on the objects within the bucket. In my case, this was for anyone (\*) and for any object (\*) within the bucket. So, now our bucket policy contains permissions for both CloudFront and S3 bucket itself.

placeholder

Compared to the permission settings for my CloudFront distribution, using S3 meant my objects in the bucket had to be exposed to the public, making my bucket less secure, and anyone stealing whatever object they wanted.

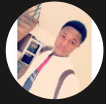


**Ahmed Tetteh**

NextWork Student

[nextwork.org](https://nextwork.org)

I preferred using CloudFront to host my website because it was the only resource allowed to retrieve my website contents from the bucket or origin.

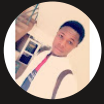


## S3 vs CloudFront Load Times

Load time means the time it takes for a browser to load contents from a website. The load times for the CloudFront site were faster than the S3 site because CloudFront's CDN capabilities allowed for my website contents to be cached at edge locations much closer to me than that of the S3 regional service.

A business would prefer CloudFront when looking to reduce the latency of their websites(load times) for users much further away. S3 static website hosting might be sufficient when we just want to do a local test, starting out, or just serve contents to users of only a specific region.





The screenshot shows a web browser window with the address bar displaying `d50mk5glghbcm.cloudfront.net`. The page content is a simple form titled "User Information" with a text input field labeled "Enter User ID" and a "Get User Data" button. Below the browser window, the Chrome DevTools network tab is open, showing a list of network requests. The first request is a GET request to `/` with a status of 200 and a size of 564 B. The second request is a GET request to `style.css` with a status of 200 and a size of 447 B. The third request is a GET request to `script.js` with a status of 200 and a size of 400 B. The fourth request is a GET request to `favicon.ico` with a status of 200 and a size of 111 B. The network tab also shows a detailed view of the first request, including the request headers, cookies, and a timing diagram.

| Status | Method | Domain                       | File        | Initiator  | Type | Transferred | Size  |
|--------|--------|------------------------------|-------------|------------|------|-------------|-------|
| 200    | GET    | d50mk5glghbcm.cloudfront.net | /           | document   | html | cached      | 564 B |
| 200    | GET    | d50mk5glghbcm.cloudfront.net | style.css   | styleSheet | css  | cached      | 447 B |
| 200    | GET    | d50mk5glghbcm.cloudfront.net | script.js   | script     | js   | cached      | 400 B |
| 200    | GET    | d50mk5glghbcm.cloudfront.net | favicon.ico | img        | xml  | cached      | 111 B |



[nextwork.org](https://nextwork.org)

# The place to learn & showcase your skills

Check out [nextwork.org](https://nextwork.org) for more projects

